

Full Documentation — es-test: Secure 3-Node ElasticSearch Cluster on AWS

Classification: INTERNAL

Infrastructure Engineer take-home test. Repo: `es-test/` (Terraform + Ansible), region `ap-southeast-3`.

Table of Contents

1. Overall Architecture & Workflow
2. Source Code Explanation — Stack by Stack
3. How to Use & Integrate
4. Monitoring & Alerting — Implementation and Flow
5. How to Debug Issues
6. Failover Mechanism

1. Overall Architecture & Workflow

This repo consists of **4 independent stacks**, each with its own `terraform/` + `ansible/` + `deploy.sh`, and they must be applied **in order** because downstream stacks read upstream state via `terraform_remote_state` (local backend, reading `.tfstate` files directly):

- | | |
|-------------------------------------|--|
| 1. <code>es-test/</code> | -> 3-node ElasticSearch cluster (foundation: VPC, SG, KMS, Secrets Manager, EC2) |
| 2. <code>es-test/vpn/</code> | -> Pritunl VPN server (reads VPC/SG from #1) |
| 3. <code>es-test/monitoring/</code> | -> Kibana + dashboard (reads ES node IPs from #1, VPN SG from #2) |
| 4. <code>es-test/alerting/</code> | -> CloudWatch alarms + SNS + cron metric pusher (reads instance IDs from #1) |

Each stack's `deploy.sh` follows the same identical 3-step pattern: 1. `terraform apply` — create/update AWS resources. 2. Auto-generate `ansible/inventory.ini` from `terraform output -json` (never hand-edited). 3. `ansible-playbook` — install & configure software on the instance, connection **always via SSM Session Manager** (`ansible_connection=community.aws.aws_ssm`), **never SSH**.

Core design principles repeated across every stack: - **Zero SSH** — all instance access goes through AWS SSM, automatic audit trail via CloudTrail, no port 22 open anywhere. - **No hardcoded secrets** — every password is generated by Terraform (`random_password`), stored in AWS Secrets Manager (encrypted at rest with a KMS CMK), fetched by the instance at runtime via a per-secret-scoped IAM role. - **Idempotent** — every playbook is safe to re-run (guarded via marker blocks, cert SAN checks, `creates:` on files). - **Fail-loud** — playbooks are designed to fail explicitly when something is wrong, instead of silently reporting "success" while actually broken (see bug details in Section 5).

DIAGRAM 1 — 4-STACK PROVISIONING ORDER (DEPENDENCY VIA TERRAFORM_REMOTE_STATE)

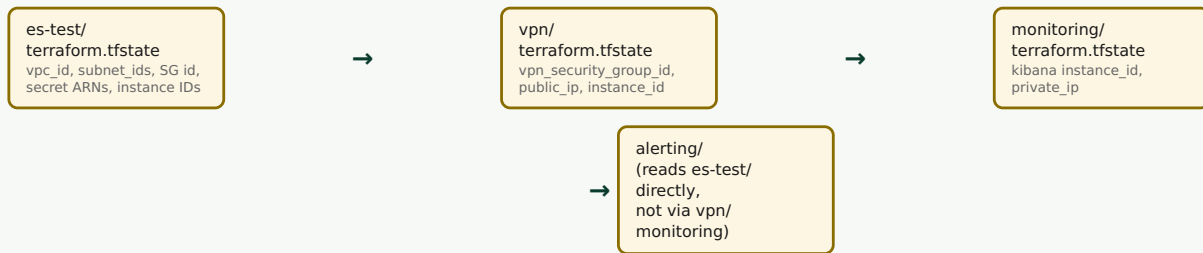


DIAGRAM 2 — DEPLOY.SH PATTERN (IDENTICAL IN EVERY STACK)



2. Source Code Explanation — Stack by Stack

DIAGRAM 3 — WHAT EACH STACK READS FROM OTHERS (DATA FLOW ACROSS TERRAFORM STATE)



2.1 Stack `es-test/` (3-node ElasticSearch, foundation for every other stack)

`terraform/main.tf` — creates: - A **VPC** `10.1.0.0/16` with 1 public subnet (`10.1.0.0/24`) + 2 private subnets (`10.1.1.0/24`, `10.1.2.0/24`) across 2 different Availability Zones. - A **NAT Gateway** in the public subnet — so nodes in private subnets can still reach the internet (yum repos, AWS API) without their own public IP. - A **Security Group** (`es-test-sg`) with 3 separate rules (deliberately split out of the inline block — see a real bug note on this in Section 5): ingress 9200 from `allowed_cidr`, egress all, and a **self-referencing** ingress 9300 rule (only members of the same SG may talk to each other on the transport layer). - A **KMS Customer-Managed Key** (`es_cmek`) with key rotation enabled — used for both EBS encryption and Secrets Manager. - **3 Secrets Manager secrets**: `elastic-password`, `kibana-system-password`, `ca-bundle` (CA cert+key for inter-node transport TLS) — all with `recovery_window_in_days=0` so a destroy/recreate cycle doesn't get blocked by AWS. - An **IAM role** (`es_role`) with scoped permissions: `secretsmanager:GetSecretValue` only on the 2 specific ARNs it needs, `kms:Decrypt` only on that CMK, plus the SSM managed core policy. - An **S3 bucket** (`ansible_ssm_transfer`) — used by the `community.aws.aws_ssm` connection plugin as a file-transfer relay (not a direct network path, still zero-SSH). - **3x EC2 instances** (`for_each`, not `count`) spread across the 2 private subnets via `% 2` — minimal `user_data`, just installs `amazon-ssm-agent` (this AMI doesn't ship it by default). ElasticSearch installation itself is **deliberately not** in `user_data` — it was moved to Ansible because `user_data` runs once at boot and doesn't know other nodes' IPs yet (a real bug that was fixed, see Section 5).

`ansible/elasticsearch.yml` — the main ES configuration playbook, key tasks: 1. Raise `vm.max_map_count` to 262144 (required by ES bootstrap checks). 2. Install ES 8.x from the official Elastic repo (`gpgcheck: true` — package signature is verified). 3. Fetch the CA bundle from Secrets Manager, generate a **per-node certificate** signed by that same CA (`elasticsearch-certutil`) — used for both HTTP TLS (9200) and transport TLS (9300). 4. Store the PKCS12 keystore password in **ES's own encrypted keystore** (`elasticsearch-keystore add -x`), never plaintext in `elasticsearch.yml`. 5. Write a `unicast_hosts.txt` file containing **every node's private IP** — this is the discovery mechanism (see Section 6). 6. Replace `elasticsearch.yml` with a fully Ansible-managed minimal config via `blockinfile` (idempotency guarded by a marker comment). 7. Start/restart ES (only restarts when config/certs actually changed — so re-runs don't force an unnecessary restart). 8. Set the `elastic` user's password via `elasticsearch-reset-password`, then **verify** that password actually authenticates (not just trusting the return code — this is a fix for a real bug, see Section 5). 9. Also set the `kibana_system` password (a dedicated service account for Kibana — ES 8.x rejects the `elastic` superuser for Kibana's own connection).

`terraform/outputs.tf` — exposes everything downstream stacks need: `instance_ids`, `node_private_ips`, `elastic_secret_arn`, `kibana_system_secret_arn`, `ca_bundle_secret_arn`, `vpc_id`, `private_subnet_ids`, `es_security_group_id`, `es_cmek_arn`, `es_iam_role_name`, `ansible_transfer_bucket`.

`deploy.sh` — orchestrates the 3 steps above. Polls for SSM registration (`aws ssm describe-instance-information`, ~5 minute timeout) before handing off to Ansible, retries the playbook run up to 3 times (handling an SSM race condition observed during testing).

2.2 Stack `es-test/vpn/` (Prituni VPN)

`terraform/main.tf` — reads `terraform_remote_state` from the ES stack to get `vpc_id`, `public_subnet_id`, `es_security_group_id`. Creates: - VPN Security Group: port 443 (admin UI) from `admin_cidr`, port 1194/UDP (OpenVPN tunnel) from `0.0.0.0/0` (authentication happens at the VPN client-profile layer, not the network ACL). - An additional rule on the **ES SG** allowing the VPN SG to reach 9200 (`source_security_group_id`, never a public CIDR). - 1x EC2 instance in the public subnet + an Elastic IP.

`ansible/pritunl.yml` — installs MongoDB + Pritunl from the official repo. **Explicitly documented risk note:** `gpgcheck: false` for the Pritunl repo because their upstream GPG key 404s on every path tried — a conscious trade-off, not an oversight (transport is still HTTPS).

`ansible/pritunl-provision.yml` — **fully-automated app-level bootstrap**, no manual UI needed: 1. Fixes an empty `mongodb_uri` bug in `/etc/pritunl.conf` (the default Pritunl install leaves this blank even though the service reports "running"). 2. Resets the admin password (`pritunl reset-password pritunl` — **the default admin username is `pritunl`, not `admin`**, confirmed via a direct MongoDB query). 3. Enables REST API auth via a direct MongoDB update (there's no CLI subcommand for this). 4. Runs `files/pritunl_provision.py` — a Python script that HMAC-signs requests to the Pritunl REST API to create an Organization + Server (UDP/1194) + User, then downloads the client `.ovpn` profile via the `/data/<org_id>/<user_id>/<server_id>.key` endpoint. 5. Stores every credential (admin password, API token/secret, org/server/user IDs, base64 `.ovpn`) in a **local HashiCorp Vault** (`kv/pritunl`) — never printed to stdout/logs (`no_log: true` on every sensitive task).

2.3 Stack `es-test/monitoring/` (Kibana + Dashboard)

`terraform/main.tf` — reads remote state from both ES **and** VPN. Creates: - Kibana Security Group: port 5601 only from the **VPN SG** (`source_security_group_id`) — Kibana is **never** reachable directly from the internet, only via the VPN. - An additional rule on the ES SG allowing Kibana to reach 9200. - A Kibana IAM role with read access to 2 secrets (`kibana_system` password + CA bundle) + `kms:Decrypt` . - 1x EC2 instance in the same **private** subnet as the ES nodes (doesn't need a public IP at all).

`ansible/kibana.yml` — installs Kibana, trusts the ES CA (copies `ca_cert` into `/etc/kibana/certs/`), configures `elasticsearch.hosts` pointing at all three ES nodes, authenticates as `kibana_system` (not `elastic` — ES 8.x rejects the superuser for this). Health-checks `GET /api/status` until it's truly HTTP 200 (no tolerance for 503 — see the bug note on this in Section 5).

`ansible/kibana-dashboards.yml` (added later, at an explicit follow-up request to populate what was initially an empty Kibana) — provisions **Saved Objects** (1 index pattern + 5 visualizations + 1 dashboard) via the Kibana Saved Objects Import API (`POST /api/saved_objects/_import?overwrite=true`), using the `elastic` user (not `kibana_system` — that service account is scoped for internal Kibana→ES traffic only and cannot authenticate against Kibana's own API layer). The ndjson bundle is static (`files/kibana-saved-objects.ndjson` , checked into the repo, no sensitive data) and defines the "ES Test Cluster Overview" dashboard (5 panels: cluster health, per-node CPU/heap/disk, doc count, node count). It's also set as Kibana's **default landing page** (`defaultRoute` setting) — so logging in immediately shows a populated view.

2.4 Stack `es-test/alerting/` (CloudWatch + SNS + Cron Metric)

`terraform/main.tf` — reads remote state from ES. Creates: - An SNS topic `es-test-alerts` + email subscription (**the single manual step in the entire system** — AWS has no API to auto-confirm an email subscription). - **7 CloudWatch alarms**: `CPUUtilization>80%` (per node, 15 minutes) and `StatusCheckFailed>0` (per node, 2 minutes) — native, no agent needed. Plus **1 custom alarm** `ClusterHealthStatus>0` tied to the custom metric described below. - Every alarm uses `treat_missing_data = "breaching"` — if data goes missing (instance dead/cron stopped), that is **treated as an alarm**, not silence. - An additional IAM policy (`cloudwatch:PutMetricData` scoped to namespace `EsTest/Custom`) attached to the existing ES role (not a new role).

`ansible/es-metrics-cron.yml` — installs a cron job (every 1 minute) running on each ES node that does **two things at once**: 1. Curls `_cluster/health` locally → maps status to a number (`green=0, yellow=1, red=2`) → `aws cloudwatch put-metric-data` into namespace `EsTest/Custom` — this is the basis for the cluster-health alarm. 2. **Self-indexes** a metrics document (status, CPU%, heap%, free disk%, doc count — pulled from `_nodes/stats` and `_count`) into the `es-test-metrics-<date>` index on ES itself — this is the data source for the Kibana dashboard. Wrapped in `|| true` so a failure in this step never breaks the CloudWatch push, which is the critical path for alerting. Also creates an **index template** (`es-test-metrics-*`) with explicit field mappings **before** the first cron run — so Kibana visualization field types are correct (not left to auto-detect, which sometimes guesses wrong).

3. How to Use & Integrate

DIAGRAM 4 — OPERATOR PROVISIONING ORDER (COMMAND-LEVEL)

1. `cd es-test/ && ./deploy.sh <allowed_cidr>`
ElasticSearch 3-node



2. `cd vpn/ && ./deploy.sh <admin_cidr>`
Pritunl VPN



3. `cd monitoring/ && ./deploy.sh`
Kibana + dashboard

3.1 Prerequisites

- AWS credentials with full access to VPC/EC2/IAM/KMS/SecretsManager/CloudWatch/SNS/S3/SSM in the target account.
- `terraform`, `ansible`, AWS CLI, and `session-manager-plugin` already installed and on `PATH`.
- No SSH keypair needed at all.

3.2 Provisioning Order (must be sequential) ↓

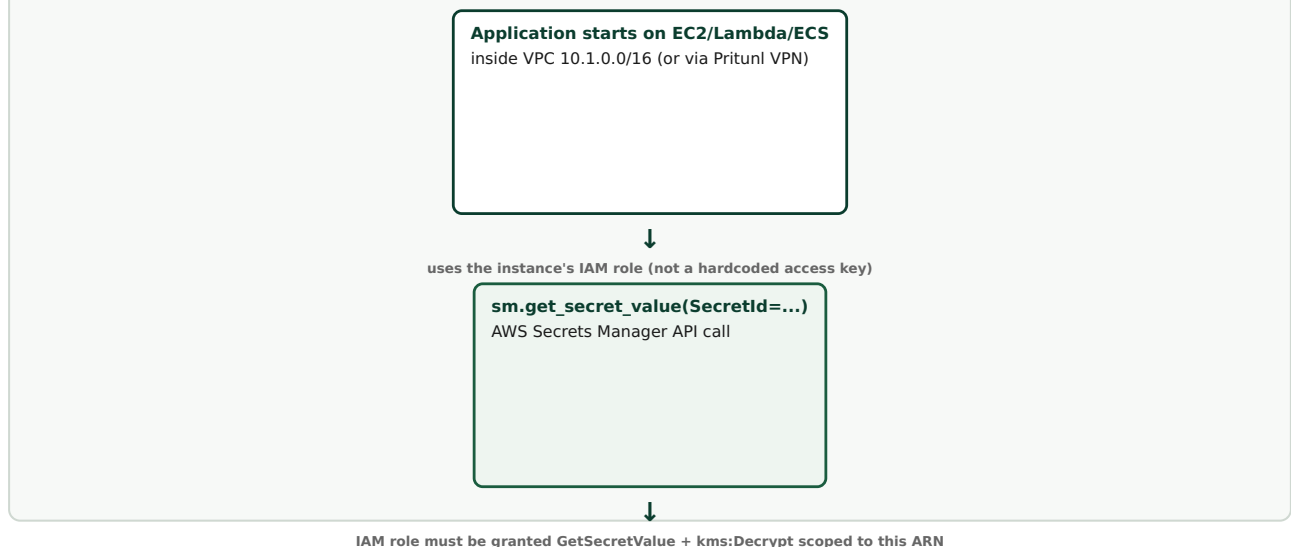
4. `cd alerting/ && ./deploy.sh <alert_email>`

1. es-test/	<code>./deploy.sh <allowed_cidr></code>	# ElasticSearch 3-node
2. es-test/vpn/	<code>./deploy.sh <admin_cidr></code>	# Pritunl VPN (reads ES state)
3. es-test/monitoring/	<code>./deploy.sh</code>	# Kibana + dashboard (reads ES+VPN state)
4. es-test/alerting/	<code>./deploy.sh <alert_email></code>	# CloudWatch + SNS (reads ES state)

Teardown order is the reverse (alerting → monitoring → vpn → es-test), due to the `terraform_remote_state` dependency chain.

3.3 Application Integration with ES (not hardcoded)

DIAGRAM 5 — HOW AN APPLICATION FETCHES THE ES PASSWORD (RUNTIME, NOT HARDCODED)



This repo is purely infrastructure provisioning — there's no sample application code. The intended integration pattern for any client application:

1. **Network placement:** the application must run inside the same VPC (10.1.0.0/16), or connect via the Pritunl VPN. ES is **never** exposed to the public internet.
2. **Credentials fetched at runtime, not hardcoded:** the compute running the app (EC2/Lambda/ECS) needs a new IAM role granted `secretsmanager:GetSecretValue` on `elasticsearch/es-test/elastic-password` + `kms:Decrypt` on the CMK — scoped the same way the existing ES/Kibana roles are. This role **doesn't exist in the repo yet** since there's no real consumer app.

3. Example pattern (Python): ``python3 -m boto3 from elasticsearch import Elasticsearch

```
sm = boto3.client("secretsmanager", region_name="ap-southeast-3") password = sm.get_secret_value( SecretId="elasticsearch/es-test/elastic-password" )["SecretString"]
```

```
es = Elasticsearch( ["https://10.1.1.30:9200", "https://10.1.2.82:9200", "https://10.1.1.156:9200"], basic_auth=("elastic", password), verify_certs=False, # or ca_certs= for full verification ) ``
```

4. The password only lives in the application process's memory for the duration of the runtime — it is never written to a config file, baked into an image, or committed to source control.

3.4 Accessing Kibana (operator/reviewer)

```
http://<kibana-private-ip>:5601
```

Only reachable after connecting to the Pritunl VPN. Log in as `elastic` with the password from Secrets Manager. On login you're taken straight to the "ES Test Cluster Overview" dashboard — no manual setup required.

3.5 Verifying Cluster Health

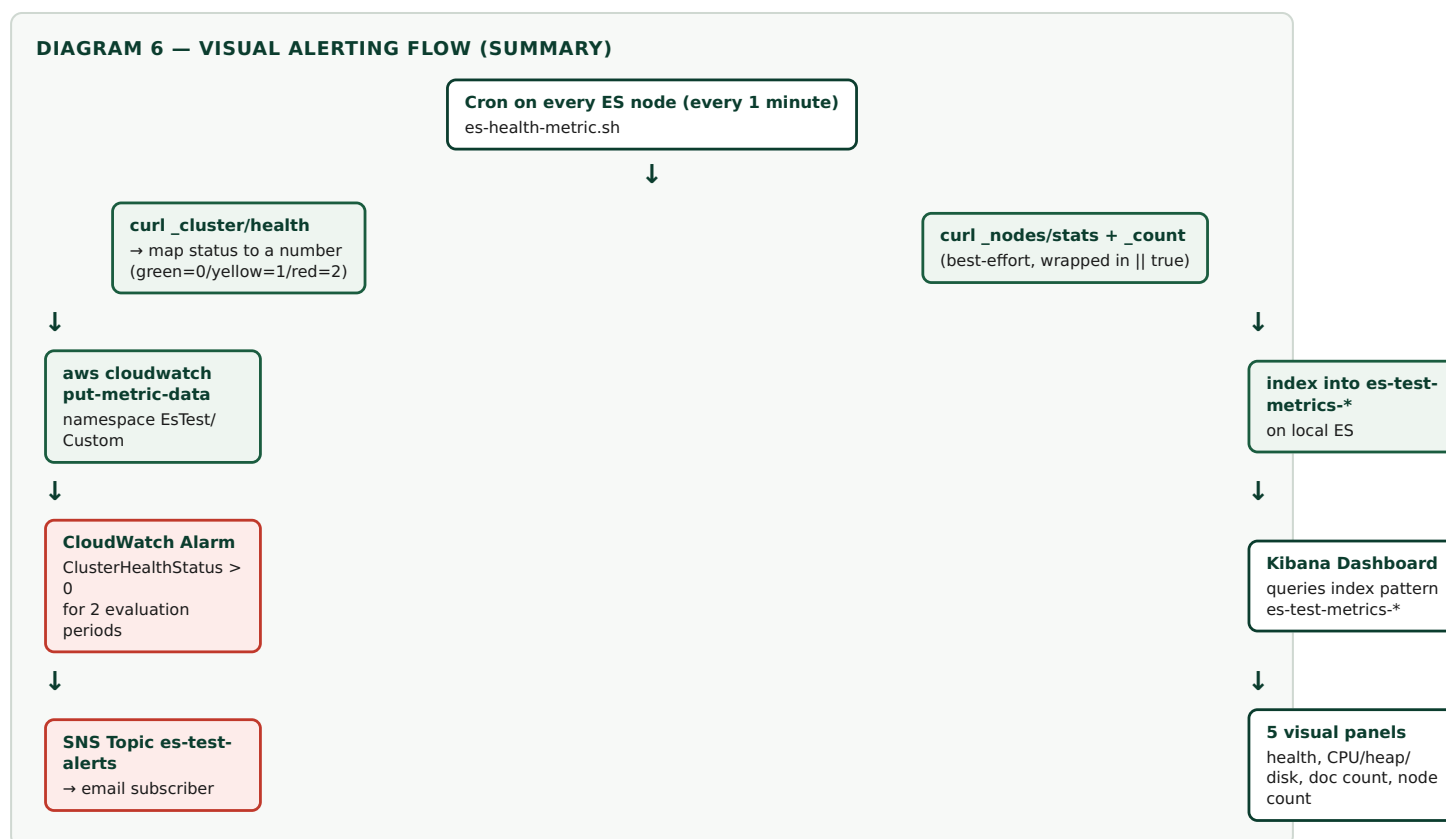
```
SECRET=$(aws secretsmanager get-secret-value --secret-id elasticsearch/es-test/elastic-password --query SecretString --output text)
curl -k -u elastic:$SECRET https://<node-ip>:9200/_cluster/health?pretty
# expect: "number_of_nodes": 3, "status": "green"
```

4. Monitoring & Alerting — Implementation and Flow

4.1 Monitoring Layers

Layer	Mechanism	What it detects
Host/instance (native)	CloudWatch <code>CPUUtilization</code> , <code>StatusCheckFailed</code>	EC2/hardware/network-level problems — no agent needed
ES internal cluster health	Cron on each node → <code>_cluster/health</code> → custom CloudWatch metric (<code>EsTest/Custom</code>)	CloudWatch itself has no visibility into ES's internal green/yellow/red status — this cron is the bridge that provides it
Visual/ dashboard	Kibana (data from <code>es-test-metrics-*</code> , populated by the same cron)	Per-node CPU/heap/disk, doc count, node count — visualized, not just a binary alarm

4.2 End-to-End Alerting Flow



Full flow detail (ASCII format, including the parallel native CloudWatch path):

```
[Every ES node, every 1 minute]
es-health-metric.sh
├── curl https://localhost:9200/_cluster/health (auth: elastic)
│   ├── map status → number (green=0/yellow=1/red=2)
│   └── aws cloudwatch put-metric-data (namespace EsTest/Custom, metric ClusterHealthStatus)
│       └── [CloudWatch Alarm "es-test-cluster-health-not-green"] evaluates >0 over 2 periods
│           └── ALARM state → SNS topic "es-test-alerts" → subscriber email
└── (best-effort, wrapped in || true) curl _nodes/stats + _count
    ├── index a metrics document into es-test-metrics-<date> on local ES
    └── [Kibana dashboard "ES Test Cluster Overview"] queries index pattern es-test-metrics-*
        └── renders 5 visual panels (health, CPU/heap/disk, doc count, node count)

[In parallel, native CloudWatch, no cron involved]
AWS watches each EC2 instance directly
```

```
└─ CPUUtilization > 80% for 15 minutes → Alarm → SNS → email
└─ StatusCheckFailed > 0 for 2 minutes → Alarm → SNS → email
```

Why two separate alerting paths (native EC2 + custom cron)? CloudWatch natively only knows the instance's condition (CPU, alive/dead) — it is completely blind to ES's internal state (whether shards are allocated, whether a quorum has formed, etc). This small cron is the only bridge that gives CloudWatch application-level visibility.

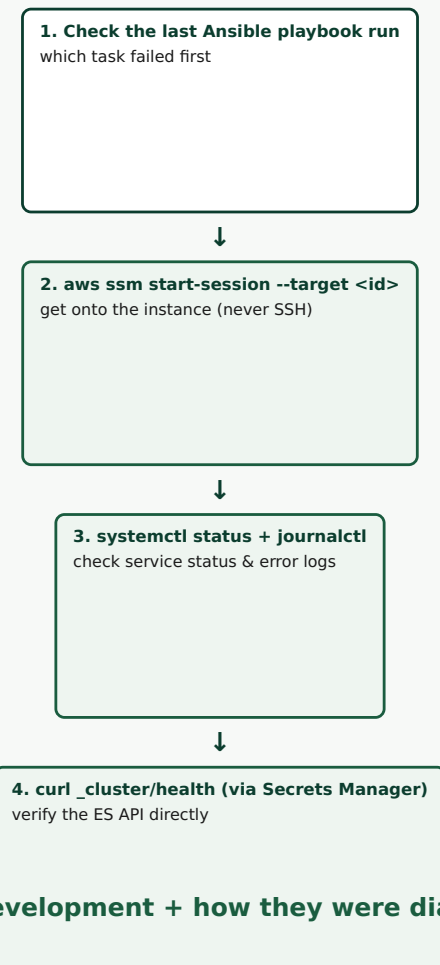
Why `treat_missing_data="breaching"` on every alarm? So that the absence of data (e.g. the cron stops running, or the instance is genuinely unreachable so no metric arrives at all) is **treated as an alarm condition**, not interpreted as "no problem because there's no data". AWS's default is `notBreaching` — which is dangerous for total-failure scenarios (silent death).

4.3 Why the Dashboard Looks Like This (not Metricbeat)

The design choice here was a manual cron+curl approach (not Metricbeat/an official Elastic agent), because it's sufficient to prove the end-to-end alerting pipeline within the scope of a take-home test, is fully auditable (one bash file that can be read end to end), and doesn't add a new service installation on every node. Trade-off: the metrics collected are shallow (no GC pause time, no rejected thread pool counts, no shard-level detail) — for real production use, this would be the first thing replaced with Metricbeat or the CloudWatch Agent's ES integration.

5. How to Debug Issues

DIAGRAM 7 — GENERAL DIAGNOSIS FLOW (STEPS 1-8, SEE SECTION 5.2)



5.1 Real bugs found during development + how they were diagnosed (precedent for similar issues)

Symptom	Root cause	How it was found	Fix
ES nodes never form a cluster (number_of_nodes: 1 forever)	unicast_hosts.txt on log + CloudWatch never populated with peer IPs — Terraform's user_data only knows its own IP at boot time	elasticsearch/ unicast_hosts.txt on each node — if it's just 1 line (its own IP), that's the cause	Moved populating this file to Ansible (elasticsearch.yml task "Populate discovery-file") — Ansible knows every node from the inventory up front, not just itself
ES fails to start entirely, bootstrap check error	Default vm.max_map_count (65530) too low once xpack.security.enabled is enabled	journalctl -u elasticsearch -t, safe	sysctl vm.max_map_count=262144 before install
elasticsearch-reset-password reports success (rc=0) but the password stored in Secrets Manager can't actually log in	Shell injection: the random password contained a \$ followed by a letter/digit (e.g. ...\$a4...) — when splatted directly into a shell string via Jinja, bash silently interpreted it as variable expansion and mangled/dropped it	Not visible from the logs (rc=0 , "successfully reset") — only caught by an explicit auth-check verification failing	Pass the password via Ansible's environment: (ELASTIC_PW), referenced as "\$ELASTIC_PW" inside single quotes in the printf string — instead of splatting a raw literal into the shell string
Kibana stuck on HTTP 503 forever, but the playbook still reported "success"	Same bug as above, for kibana_system — the password applied to ES didn't match what was stored in Secrets Manager (because of the same shell-injection bug),	Manually curling http://localhost:5601/api/status long after "deploy finished" — only then was it clear Kibana was actually down for good	(a) fix the same shell-injection bug, (b) change the health check to only accept status_code 200 — retry until that's reached, fail loudly if it never is

Symptom	Root cause	How it was found	Fix
	but the old health-check task accepted 503 as a pass (treated as "still starting")		
The self-referencing transport (9300) security group rule kept disappearing every time <code>allowed_cidr</code> was re-applied	Mixing inline <code>ingress / egress</code> blocks on <code>aws_security_group</code> with separate <code>aws_security_group_rule</code> resources for the same SG — Terraform's inline block overwrites all rules on that group on every apply that touches it	<code>terraform plan</code> diff showed the self-referencing transport rule being destroyed even though it wasn't requested	Remove all inline ingress/egress from the <code>aws_security_group</code> resource, move everything to separate <code>aws_security_group_rule</code> resources
Destroying and re-applying Secrets Manager fails (<code>InvalidRequestException: already scheduled for deletion</code>)	AWS's default 30-day recovery window blocks re-creating a secret with the same name	Explicit error from the AWS API	<code>recovery_window_in_days = 0</code> on every secret (a trade-off specific to a demo/test environment, not appropriate for real production secrets)
Pritunl service reports "running" but every admin/API operation fails	<code>mongodb_uri</code> empty in <code>/etc/pritunl.conf</code> even though both <code>mongod</code> & <code>pritunl</code> report active status	<code>pritunl get-mongodb</code> returns empty	<code>pritunl set-mongodb mongodb://localhost:27017/pritunl</code> then restart the service
Logging in as Pritunl admin with username <code>admin</code> always fails	Pritunl's default superuser username is actually <code>pritunl</code> , not <code>admin</code>	Queried MongoDB directly: <code>db.administrators.find()</code>	Use <code>pritunl</code> as the username in every task that resets the password/enables the API

5.2 General Debugging Steps (for new issues not already listed above)

1. **Check the status of the most recent Ansible playbook run** — every sensitive task uses `no_log: true`, so sensitive output never leaks into the log, but non-sensitive tasks (health checks, debug messages) are still recorded — read the task order to see what failed first.
2. **Get onto the instance via SSM** (never SSH): `bash aws ssm start-session --target <instance-id> --region ap-southeast-3`
3. **Check service status & logs:**
`bash sudo systemctl status elasticsearch # or: kibana, pritunl sudo journalctl -u elasticsearch -n 100 --no-pager`
4. **Verify cluster health directly from the node** (avoid printing the password — fetch it from Secrets Manager via the AWS CLI, never hardcode it): `bash SECRET=$(aws secretsmanager get-secret-value --secret-id elasticsearch/es-test/elastic-password --region ap-southeast-3 --query SecretString --output text) curl -k -u elastic:$SECRET https://localhost:9200/_cluster/health?pretty`
5. **Verify the cron metric is running** (for monitoring/alerting issues): `bash cat /var/log/es-health-metric.log # output of every cron run sudo crontab -l # confirm the entry exists`
6. **Check the CloudWatch alarm state directly:** `bash aws cloudwatch describe-alarms --region ap-southeast-3 --alarm-name-prefix es-test`
7. **Re-run the relevant playbook** — every playbook in this repo is idempotent, safe to re-run with no side effects (it won't restart the service/regenerate certs unless something actually changed).
8. **If you need a full re-provision:** re-run `deploy.sh` for the relevant stack from the root, in dependency order (see Section 3.2).

6. Failover Mechanism

Failover in this system is layered across **3 independent levels**:

DIAGRAM 8 — 3 FAILOVER LAYERS (INDEPENDENT, COMPLEMENTARY)

Layer 1: ES Cluster (native)

2/3 quorum, shard replication,
spread across 2 AZs, mTLS transport

Layer 2: Infra (CloudWatch)

StatusCheckFailed,
CPUUtilization>80%

Layer 3: Application (custom cron)

ClusterHealthStatus,
treat_missing_data=breaching

6.1 ES Cluster Layer (native)

- All 3 nodes are **master-eligible + data** (`node.roles: [master, data]`) — no read/write differentiation per node, all symmetric.
- `cluster.initial_master_nodes` registers all three nodes from the start — a **2-of-3** majority quorum is required to elect a new master if 1 node dies.
- Shard replication across nodes happens automatically (ES native) — data isn't lost if 1 node goes down, as long as a shard's primary and replica aren't both on the node that died.
- Nodes are spread across **2 different Availability Zones** (`% 2` distribution) — AZ-level fault tolerance, not just node-level.
- The transport layer (9300) uses **mutual TLS** between nodes (shared CA) — specifically required for multi-node clusters (a single-node cluster wouldn't need this).

6.2 Infra Layer (native CloudWatch)

- `StatusCheckFailed` alarm (2 minutes) — detects EC2-level hardware/network failures that AWS itself detects, independent of whether ES is still responding.
- `CPUUtilization>80%` (15 minutes) — an early signal that a node is overloaded before it actually dies.

6.3 Application Layer (custom cron → CloudWatch → SNS)

- Each node's cron polls its own `_cluster/health` — if status isn't `green`, the `ClusterHealthStatus` alarm fires, signaling a problem at the cluster-logic level (unassigned shards, split brain, etc) that the infra layer alone can't see.
- `treat_missing_data="breaching"` — a node that's truly dead (cron fully stopped) **also** triggers the alarm, instead of silently disappearing.

6.4 Minimal/Zero-Downtime Node Replacement Procedure (manual, documented, not automated)

DIAGRAM 9 — MINIMAL-DOWNTIME NODE REPLACEMENT ORDER

1. Freeze shard allocation
cluster.routing.allocation.enable=none



2. Stop/terminate/replace the instance
Terraform: taint + apply



3. New node rejoins automatically
Ansible re-provisioning (new cert + unicast_hosts.txt)

```
# 1. Freeze shard allocation before touching the node (avoid pointless reshuffling)
curl -k -u elastic:$SECRET -X PUT https://<node-ip>:9200/_cluster/settings \
  -d '{"transient":{"cluster.routing.allocation.enable":"none"}}'

# 2. Stop/terminate/replace the target instance (Terraform: taint + apply)

# 3. New node rejoins automatically (re-running deploy.sh's Ansible stage generates
#   a fresh cert + fresh unicast_hosts.txt, idempotently)

# 4. Re-enable allocation
curl -k -u elastic:$SECRET -X PUT https://<node-ip>:9200/_cluster/settings \
  -d '{"transient":{"cluster.routing.allocation.enable":"all"}}'

# 5. Wait for status to return to green before moving on to the next node (rolling, one at a time, NOT parallel)
```

Because quorum is maintained throughout this procedure (2 of 3 nodes are always alive), the cluster remains **fully available** (green→yellow→green per node), not a full outage.

What's intentionally not automated (scope boundary of the take-home test): the procedure above hasn't been wired into a single Terraform/Ansible command yet — a real production setup would ideally use an Auto Scaling Group with an `instance_refresh` policy.

This document is a complete technical summary of the `es-test/` repo. For the take-home test's literal required answers (solution description, resources consulted, time spent, feedback), see `docs/02_SUPPORTING_INSTRUCTIONS_EN.md` in the repo.